

ZeroDDS

DDS-TS 1.0

DDS-TS

TypeScript Platform-Specific Model for OMG IDL 4.2

Version `zerodds/v1.0-beta1`

Status: Beta

Date: 2026-04-29

Editors: Sandra Keßler

Copyright © 2026 ZeroDDS Project. All rights reserved.

This specification is published under the ZeroDDS Vendor Specification Licence.

Contents

Status of this Document	4
1 Scope	5
1.1 Purpose	5
1.2 Out of Scope	5
2 Conformance	6
2.1 IDL-Codegen Profile	6
2.2 Decorator-Runtime Profile	6
2.3 WASM-Bindings Profile	7
3 Normative References	8
4 Terms and Definitions	9
5 Symbols and Abbreviations	10
6 Overview (Informative)	11
6.1 Mapping Philosophy	11
6.2 Toolchain	11
6.3 Code-Style Toggle	11
7 Type Mapping (Normative)	12
7.1 Primitive Types	12
7.2 Struct Mapping	12
7.3 Union Mapping	13
7.4 Enum Mapping	13
7.5 Bitset Mapping	13
7.6 Bitmask Mapping	14
7.7 Sequence and Array Mapping	14
7.7.1 Bounded and Unbounded Sequences	14
7.7.2 Fixed-Size Arrays	15
7.7.3 Multi-Dimensional Arrays	15
7.7.4 Element Mutability	15
7.7.5 Empty Sequences	15
7.8 Typedef Mapping	16
7.8.1 Simple Type Aliases	16
7.8.2 Sequence Aliases	16
7.8.3 Array Aliases	16
7.8.4 Recursive Resolution	16

7.8.5	Forward References	17
7.9	Optional and Default Values	17
7.9.1	Optional Fields	17
7.9.2	Encoding of Absent Values	17
7.9.3	Default Values	17
7.10	Annotation Mapping	18
7.10.1	Mapping Table	18
7.10.2	Annotation Resolution Order	19
7.10.3	Annotation Pass-Through to TSDoc	19
8	Decorator Runtime (Normative)	20
8.1	Class Decorators	20
8.2	Field Decorators	20
8.3	Reflection API	20
9	Verbatim Code-Generation Hook	22
9.1	Purpose	22
9.2	Annotation Form	22
9.3	Placement Kinds	22
9.4	Quoting and Escaping	23
9.5	Multi-Line Text	23
9.6	Multiple Annotations on One Declaration	23
9.7	Reference Implementation Footprint	23
10	Code Style (Informative)	25
10.1	Identifier Casing	25
10.2	Module Structure	25
10.3	Output Format	25
	Annex A (informative): IDL Examples and Generated Output	26
	Annex B (normative): Compliance Test Suite	28
	Annex C (normative): WASM-Bindings Profile	29

Status of this Document

This is a Beta release of the DDS-TS Vendor Specification, version `zerodds/v1.0-beta1`, published by the ZeroDDS Project. It is made available for public review and feedback prior to Final publication.

This specification defines a Platform-Specific Model (PSM) that maps the OMG Interface Definition Language (IDL) version 4.2 to TypeScript. It is designed as a peer to the existing OMG-defined language PSMs (IDL-to-C++, IDL-to-C#, IDL-to-Java) and follows the same structural conventions: a normative type-mapping table, a normative annotation-mapping table, and explicit conformance profiles.

The TypeScript language definition is governed by ECMA-262 (ECMAScript Language Specification) and the TC39 TypeScript Language Specification proposal; this document does not redefine either, but specifies how IDL constructs map onto TypeScript constructs.

The terms `MUST`, `MUST NOT`, `SHALL`, `SHALL NOT`, `SHOULD`, `SHOULD NOT`, `MAY`, `MAY NOT`, `REQUIRED`, `RECOMMENDED`, and `OPTIONAL` are to be interpreted as described in IETF RFC 2119 and RFC 8174.

1 Scope

1.1 Purpose

This specification defines a TypeScript Platform-Specific Model (PSM) for the OMG Interface Definition Language version 4.2, with extensions for the Extensible and Dynamic Topic Types for DDS (X-Types) version 1.3 annotations.

The mapping enables IDL types defined for DDS topics to be generated as TypeScript declarations and consumed in browser-side or Node.js-side TypeScript code without loss of semantic information.

1.2 Out of Scope

The following items are explicitly outside the scope of this specification:

- TypeScript runtime support beyond the decorator-runtime library defined in Chapter 8.
- DDS data wire encoding (XCDR2): the present specification produces TypeScript type declarations only; serialization is the responsibility of a separate runtime implementation (e.g. a WebAssembly build of the ZeroDDS core, see § 10.3).
- Bindings to a specific JavaScript runtime: this specification produces ECMAScript-Module (ESM) source that is compatible with any TypeScript-supporting build tool (Vite, esbuild, tsc, swc, etc.).

2 Conformance

This specification defines three conformance profiles. A conformance claim **MUST** cite this specification by full identifier (DDS-TS, `zerodds/v1.0-beta1` or later) and enumerate every profile to which conformance is claimed.

2.1 IDL-Codegen Profile

Conformance: IDL-Codegen Profile

An implementation conforms to the IDL-Codegen Profile if and only if all of the following are true:

1. It implements the type mapping defined in Chapter 7 for every primitive, constructed, and template type of OMG IDL 4.2.
2. It implements the annotation mapping defined in §7.10 for every X-Types 1.3 annotation.
3. It produces output that is valid TypeScript ≥ 5.0 source code, compilable with `tsc -strict`.
4. It satisfies the conformance test cases in Annex B.

2.2 Decorator-Runtime Profile

Conformance: Decorator-Runtime Profile

An implementation conforms to the Decorator-Runtime Profile if and only if all of the following are true:

1. It provides class-level decorators `@Final`, `@Appendable`, `@Mutable`, `@DdsTopic` with the semantics defined in Chapter 8.
2. It provides field-level decorators `@Key`, `@Id`, `@Optional` with the semantics defined in Chapter 8.
3. It provides reflection functions `getDdsType`, `getDdsKey`, `getDdsTopic`.

2.3 WASM-Bindings Profile

Conformance: WASM-Bindings Profile

An implementation conforms to the WASM-Bindings Profile if and only if all of the following are true:

1. It conforms to the IDL-Codegen Profile (§ 2.1) and to the Decorator-Runtime Profile (§ 2.2).
2. It exposes the DDS DCPS data-path operations enumerated in Annex C, § C.1, as TypeScript-callable functions backed by a WebAssembly module compiled from a DDS implementation (e.g. the ZeroDDS Rust core).
3. It serialises and deserialises samples using the XCDR2 wire format on both sides of the WASM boundary; the binding layer **MUST NOT** expose JavaScript-native object representations as the canonical wire form.
4. It satisfies the conformance test cases in Annex C, § C.2 (round-trip publish/subscribe through the WASM boundary).

The design and operational semantics of the Profile are normatively specified in Annex C.

3 Normative References

Identifier	Version	Title
OMG IDL	4.2	Interface Definition Language
OMG X-Types	1.3	Extensible and Dynamic Topic Types for DDS
OMG DDS	1.4	Data Distribution Service for Real-Time Systems
OMG IDL4-Cpp	1.0	IDL to C++ Mapping (peer PSM)
OMG IDL4-Csharp	1.0	IDL to C# Mapping (peer PSM)
OMG IDL4-Java	1.0	IDL to Java Mapping (peer PSM)
ECMA-262	14th ed.	ECMAScript Language Specification
TC39 TypeScript	—	TypeScript Language Specification
IETF RFC 2119	—	Key words for use in RFCs
IETF RFC 8174	—	Ambiguity of Uppercase vs Lowercase

4 Terms and Definitions

IDL	OMG Interface Definition Language as defined in <i>OMG IDL</i> 4.2.
Codegen	An implementation conforming to the IDL-Codegen Profile (§ 2.1).
Decorator	A TypeScript expression evaluated at class- or field-declaration time, attached to the declaration via the <code>@Decorator</code> syntax.
ESM	ECMAScript Module syntax: <code>export</code> and <code>import</code> statements as defined in ECMA-262.

5 Symbols and Abbreviations

DDS Data Distribution Service

ESM ECMAScript Module

IDL Interface Definition Language

PSM Platform-Specific Model

TS TypeScript

WASM WebAssembly

XCDR2 Extended Common Data Representation, version 2

6 Overview (Informative)

6.1 Mapping Philosophy

The DDS-TS mapping favours *structural typing* the natural TypeScript style over the OO-style nominal typing of the C++ and Java PSMs. Each IDL type is mapped to a TypeScript construct that preserves both compile-time type safety and runtime introspection capability through the decorator-runtime library.

6.2 Toolchain

The reference implementation in `crates/idl-ts/` produces `.ts` source files compatible with TypeScript ≥ 5.0 . The generated source uses ECMAScript-Module (ESM) syntax exclusively.

6.3 Code-Style Toggle

By default, IDL identifier names are preserved verbatim (`snake_case` to `snake_case`, `Camel-Case` to `CamelCase`). An optional *camelCase-toggle* configuration (Chapter 10) re-cases `snake_case` IDL fields to `camelCase` TypeScript fields.

7 Type Mapping (Normative)

7.1 Primitive Types

IDL Type	TypeScript Type	Notes
boolean	boolean	
char / wchar	string	single-character string
octet	number	0..255
short / int16	number	
unsigned short / uint16	number	
long / int32	number	
unsigned long / uint32	number	
long long / int64	bigint	JS number is 53-bit exact
unsigned long long / uint64	bigint	
float	number	IEEE 754 binary32
double	number	IEEE 754 binary64
long double	number	implementation-defined precision
string / wstring	string	

7.2 Struct Mapping

A bounded IDL `struct` SHALL be mapped to a TypeScript `interface`. Each field of the struct SHALL be mapped to a property of the interface.

```
struct Point { long x; long y; };
```

maps to

```
export interface Point {  
  x: number;  
  y: number;  
}
```

7.3 Union Mapping

```
union Variant switch (long) {
  case 1: long i;
  case 2: string s;
};
```

maps to

```
export type Variant =
  | { discriminator: number; i: number }
  | { discriminator: number; s: string };
```

7.4 Enum Mapping

```
enum Color { RED, GREEN, BLUE };
```

maps to

```
export enum Color { RED = "RED", GREEN = "GREEN", BLUE = "BLUE" }
```

7.5 Bitset Mapping

```
bitset Flags { bitfield<3> low; bitfield<5> high; };
```

maps to

```
export interface Flags { low: number; high: number; }
export const Flags_low_BITS = 3;
export const Flags_high_BITS = 5;
```

7.6 Bitmask Mapping

```
bitmask Permissions { READ, WRITE, EXEC };
```

maps to

```
export const Permissions = {
  READ: 1 << 0,
  WRITE: 1 << 1,
  EXEC: 1 << 2,
} as const;
export type Permissions = number;
```

7.7 Sequence and Array Mapping

7.7.1 Bounded and Unbounded Sequences

7.1

```
struct Sample {
  sequence<long, 10> pixels; // bounded
  sequence<string> tags; // unbounded
};
```

maps to

```
export interface Sample {
  pixels: Array<number>;
  tags:   Array<string>;
}
export const Sample_pixels_BOUND = 10;
```

7.7.2 Fixed-Size Arrays

```
struct Vector3 { double v[3]; };
```

maps to

```
export interface Vector3 { v: Array<number>; }
export const Vector3_v_LENGTH = 3;
```

7.7.3 Multi-Dimensional Arrays

7.7.4 Element Mutability

7.7.5 Empty Sequences

7.8 Typedef Mapping

7.8.1 Simple Type Aliases

```
typedef long Counter;  
typedef string<128> TopicName;
```

maps to

```
export type Counter = number;  
export type TopicName = string;
```

7.8.2 Sequence Aliases

```
typedef sequence<octet, 1024> ByteBuffer;
```

maps to

```
export type ByteBuffer = Array<number>;  
export const ByteBuffer_BOUND = 1024;
```

7.8.3 Array Aliases

7.8.4 Recursive Resolution

7.8.5 Forward References

7.9 Optional and Default Values

7.9.1 Optional Fields

```
@final struct Frame {
  long      seq;
  @optional
  long      retry;
};
```

maps to

```
@Final()
export class Frame {
  seq!:    number;
  @Optional()  retry?: number;           // T | undefined
}
```

7.9.2 Encoding of Absent Values

7.9.3 Default Values

```
struct Config {
    @default(5000)    long   timeout_ms;
    @default("on")    string  mode;
};
```

maps to

```
export class Config {
    timeout_ms: number = 5000;
    mode:      string  = "on";
}
```

7.10 Annotation Mapping

7.10.1 Mapping Table

8

7.9

7.9

9

7.10.2 Annotation Resolution Order

- 1.
- 2.
- 3.
- 4.
- 5.

7.10.3 Annotation Pass-Through to TSDoc

```
struct Telemetry {  
    @unit("celsius") @min(-40) @max(125)  
    double temperature;  
};
```

maps to

```
export interface Telemetry {  
    /**  
     * @dds-unit celsius  
     * @dds-min -40  
     * @dds-max 125  
     */  
    temperature: number;  
}
```

8 Decorator Runtime (Normative)

8.1 Class Decorators

`@Final()`

`@Appendable()`

`@Mutable()`

`@DdsTopic(name)`

8.2 Field Decorators

`@Key()`

`@Id(N)`

`@Optional()`

8.3 Reflection API

`getDdsType(value)`

`getDdsKey(sample)`

`getDdsTopic(value)`

9 Verbatim Code-Generation Hook

9.1 Purpose

9.2 Annotation Form

```
@verbatim(language = "ts" | "*",  
           placement = <PlacementKind>,  
           text      = "<source-text>")
```

9.3 Placement Kinds

9.4 Quoting and Escaping

9.5 Multi-Line Text

9.6 Multiple Annotations on One Declaration

9.7 Reference Implementation Footprint

```
@verbatim(language = "ts",
           placement = "BEGIN_FILE",
           text      = "// Copyright Acme Corp.\n// SPDX-License-
                       Identifier: Apache-2.0")
struct AcmeMessage { long seq; };
```

produces

```
// Copyright Acme Corp.
// SPDX-License-Identifier: Apache-2.0
```

```
// Generated by zerodds idl-ts. Do not edit.  
  
export interface AcmeMessage { seq: number; }
```


10 Code Style (Informative)

10.1 Identifier Casing

10.2 Module Structure

10.3 Output Format

Annex A

(informative)

IDL Examples and Generated Output

A.1 Bounded Sequence

```
struct Telemetry {  
    string<32>          name;  
    sequence<double, 16> readings;  
};
```

```
export interface Telemetry {  
    name:      string;  
    readings: Array<number>;  
}
```

A.2 Mutable Type with Explicit Ids

```
@mutable  
struct Config {  
    @id(0) string endpoint;  
    @id(1) long  timeout_ms;  
};
```

```
import { Mutable, Id } from "@zerodds/types";  
  
@Mutable()  
export class Config {  
    @Id(0) endpoint!: string;  
    @Id(1) timeout_ms!: number;  
}
```

A.3 Bitset with Width-Constants

```
bitset RegisterMap {  
    bitfield<8>    device_id;  
    bitfield<3>    status;  
    bitfield<5>    reserved;  
};
```

```
export interface RegisterMap {  
    device_id: number;  
    status:    number;  
    reserved:  number;  
}  
export const RegisterMap_device_id_BITS = 8;  
export const RegisterMap_status_BITS    = 3;  
export const RegisterMap_reserved_BITS  = 5;
```

Annex B

(normative)

Compliance Test Suite

B.1 Codegen-Profile Tests

- 1.
2. 7.1
- 3.
- 4.
- 5.
- 6.

B.2 Decorator-Runtime Tests

1. 8
- 2.
- 3.
- 4.

B.3 Reference Test-Harness

<https://gitlab.sandra-kessler.eu/fishermen21/zerodds/>

Annex C

(normative)

WASM-Bindings Profile

C.1 Required Operations

C.1.1 Participant + Topic

C.1.2 Publisher / Subscriber

C.1.3 DataWriter / DataReader

```
function writeSample(w: DataWriterHandle, xcdr2: Uint8Array): void;
function takeSamples(r: DataReaderHandle, max: number): Sample[];

function deleteDataWriter(w: DataWriterHandle): void;
function deleteDataReader(r: DataReaderHandle): void;
```

C.1.4 Listener Callbacks

```
type DataAvailableCallback =
  (r: DataReaderHandle) => void;

function setDataAvailableListener(
  r: DataReaderHandle,
  cb: DataAvailableCallback
): void;
```

C.2 Wire-Format Boundary

C.2.1 XCDR2 Sample Bytes

C.2.2 Memory Ownership

C.3 Browser vs Node.js Bindings

C.3.1 Browser Profile

C.3.2 Node.js Profile

C.4 Conformance Tests

C.4.1 Round-Trip Publish/Subscribe

1.

2.

3.

C.4.2 Listener Callback

C.4.3 Resource Cleanup

C.5 Reservation of Future Extensions